

Getting started with the Camera Module

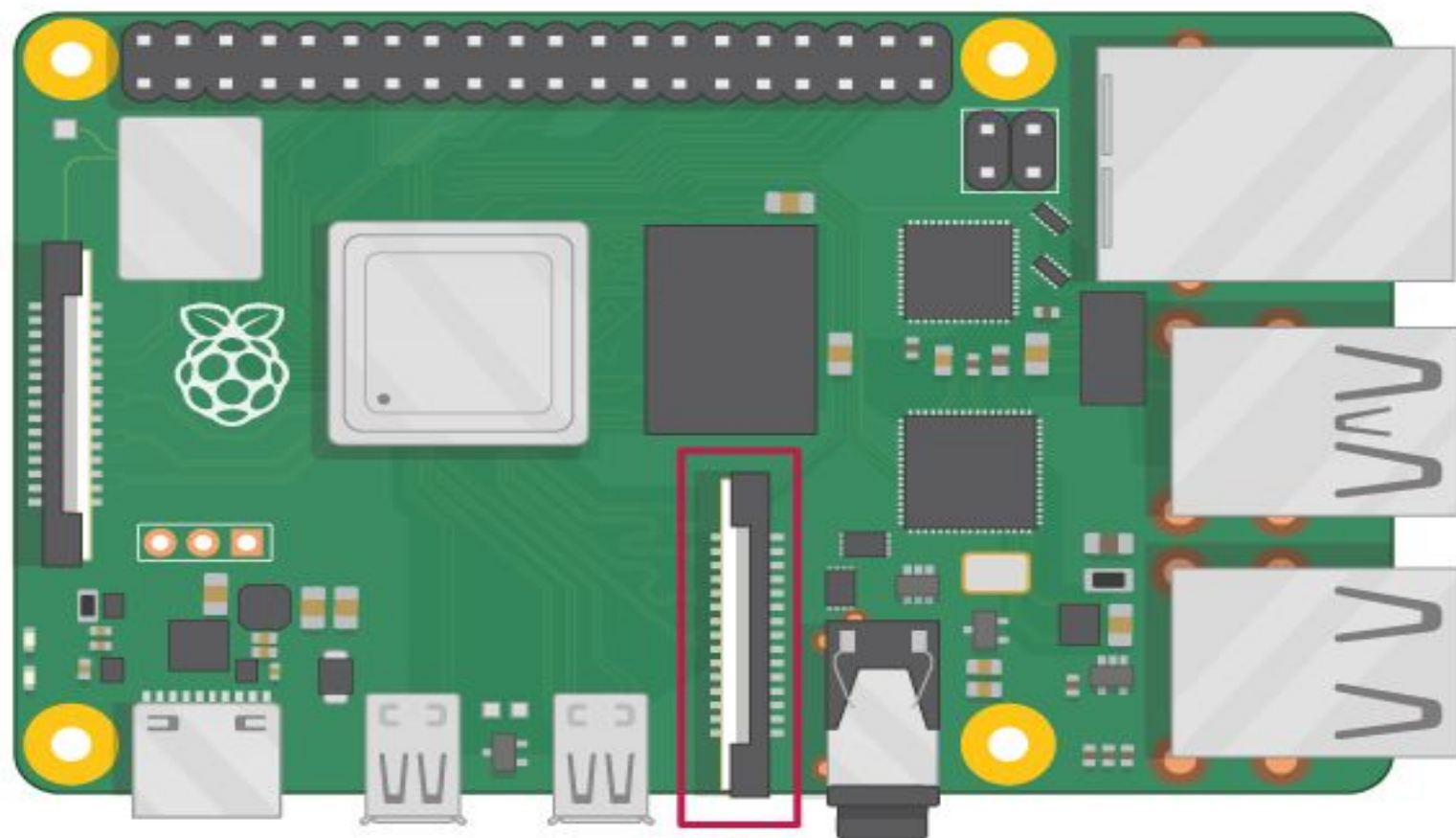


Introduction

- You will need to install picamzero a library designed to make using the camera on the Raspberry Pi as easy as possible.

What you will need

- **Raspberry Pi computer with a Camera Module port**
- All current models of Raspberry Pi have a port for connecting the Camera Module. (The Raspberry Pi 5 has two ports.)



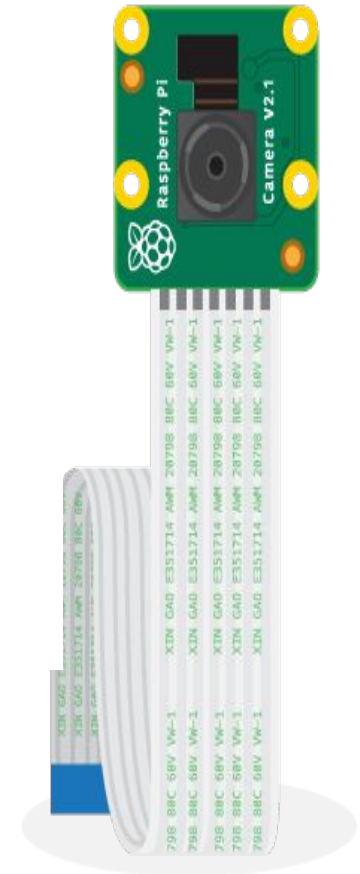
**Camera
Module port**

Raspberry Pi Camera Module

There are two versions of the Camera Module 2:

- [The standard version](#), which is designed to take pictures in normal light
- [The NoIR version](#), which doesn't have an infrared filter, so you can use it together with an infrared light source to take pictures in the dark

Note: Camera Module 3 (released in 2023) is available with a choice of standard and wide lenses, both with or without an infrared filter.



Connect the Camera Module

1. Locate the Camera Module port
2. Gently pull up on the edges of the port's plastic clip
3. Insert the Camera Module ribbon cable; make sure the connectors at the bottom of the ribbon cable are facing the contacts in the port.
4. Push the plastic clip back into place
5. Reboot your Raspberry Pi.

First Step (Run This Commands)

```
$pi: sudo apt update
```

```
$pi: sudo apt install python3-picamzero
```

```
Verify With :- apt list --installed | grep picamzero
```

How to control the Camera Module via the command line

Now your Camera Module is connected and the software is enabled, try out the command line tools `rpicas-still` and `rpicas-vid`.

Open a terminal window by clicking the black monitor icon in the taskbar:

Type in the following command to take a still picture and save it to the Desktop:

```
rpicas-still -o ~/Desktop/test.jpg
```


By adding different options, you can set the size and look of the image the `rpicas-still` command takes.

- Add `--width` and `--height` to change the dimensions of the image:

```
rpicas-still -o ~/Desktop/image-small.jpg  
--width 640 --height 480
```

- Now record a video with the Camera Module by using the following `rpicas-vid` command:

```
rpicas-vid -o ~/Desktop/video_test.mp4
```

How to control the Camera Module with Python code

The Python **picamzero** library allows you to control your Camera Module.

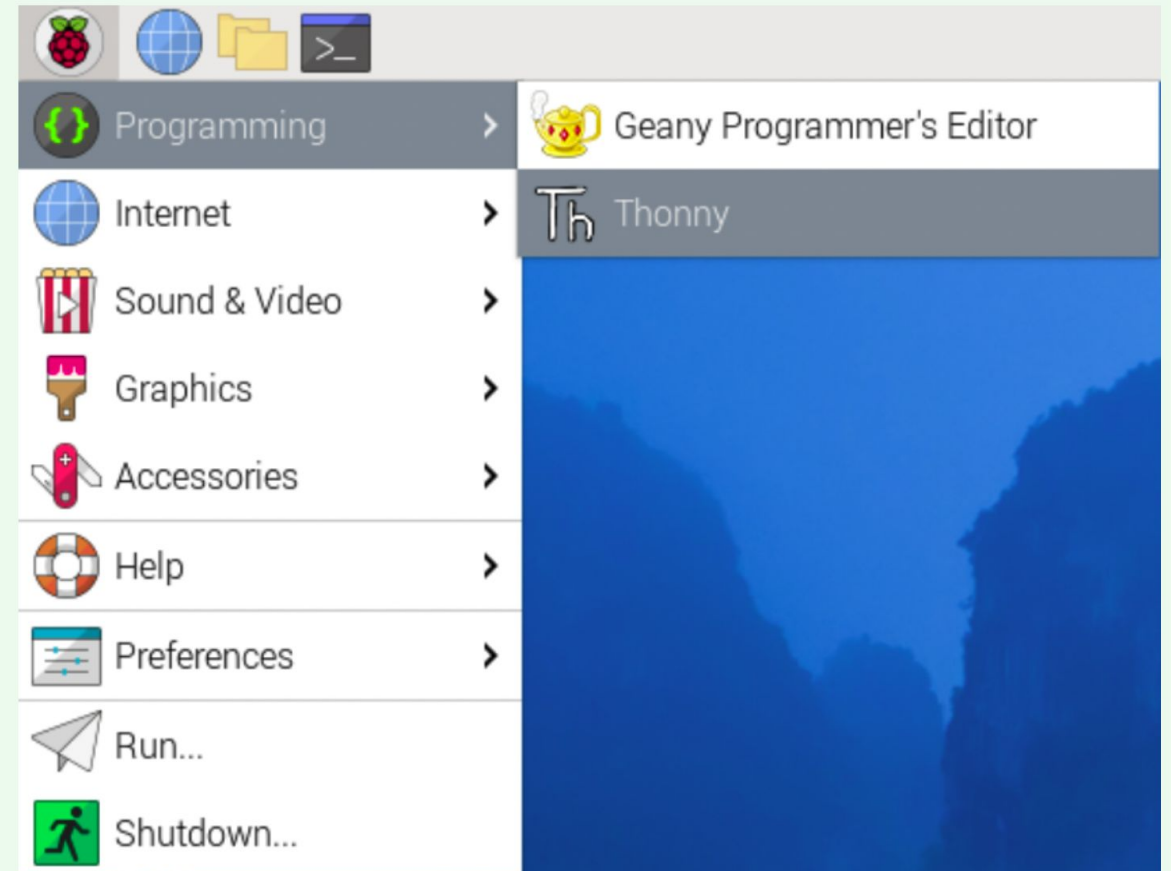
- Open a terminal and type the following commands one at a time to install picamzero:

```
$pi: sudo apt update
```

```
$pi: sudo apt install python3-picamzero
```

Open a new file and save it to your Desktop as `camera.py`

- Open a Python 3 editor, such as **Thonny**:



- **Enter the following code:**

```
from picamzero import Camera
```

```
from time import sleep
```

 - - Imports the sleep function to pause the program.

```
cam = Camera()
```

 - - Creates a Camera object.

```
cam.start_preview()
```

 - - Starts the live camera preview on the screen.

```
sleep(5)
```

 - - Keeps the preview open for 5 seconds before closing.

- Save and run your program. The camera preview should be shown for five seconds and then close again.

- If you need to rotate your preview and photos, you can use `flip_camera` with the `vflip=True` or horizontal flip: `hflip=True` arguments:

```
from picamzero import Camera
```

```
from time import sleep
```

```
cam = Camera()
```

```
cam.flip_camera(hflip=True)
```

```
cam.start_preview()
```

```
sleep(5)
```

Take still pictures with Python code

`picamzero` has a very convenient function (`take_photo`) for capturing images.

```
from picamzero import Camera
```

```
cam = Camera()
```

```
cam.start_preview()
```

```
cam.take_photo("/home/pi/Desktop/new_image.jpg")
```

```
cam.stop_preview()
```

Capture multiple images using Python.

You can also use `capture_sequence` to capture multiple images.

This code captures three images and uses a 2 second delay between each image.

```
from picamzero import Camera
cam = Camera()
cam.start_preview()
cam.capture_sequence("/home/pi/Desktop/sequence.jpg", num_images=3, interval=2)
cam.stop_preview()
```

Recording video with Python code

```
from picamzero import Camera
```

```
cam = Camera()
```

```
cam.start_preview()
```

```
cam.record_video("/home/pi/Desktop/test_video.mp4", duration=5)
```

```
cam.stop_preview()
```


Picamera Python Code Examples

Set Maximum Image Resolution

Capture images at the camera's maximum resolution.



V1 Module Max:
2592×1944 | Min: 15×15

```
from picamzero import Camera
from time import sleep
import os
```

```
home_dir = os.environ['HOME']
cam = Camera()
cam.still_size = (2592, 1944)
cam.start_preview()
```

```
sleep(2)
```

```
cam.take_photo(f'{home_dir}/Desktop/max.jpg')
cam.stop_preview()
```

Add Text Annotation

Add custom text overlay to your photos.

```
from picamzero import Camera  
import os
```

```
home_dir = os.environ['HOME']  
cam = Camera()  
cam.annotate("Hello, world!")
```

```
cam.start_preview()  
cam.take_photo(f"{home_dir}/Desktop/textOnPhoto.jpg")  
cam.stop_preview()
```

Add Timestamp

Automatically add current date/time to photos.

```
from picamzero import Camera  
from datetime import datetime  
import os
```

```
home_dir = os.environ['HOME']  
cam = Camera()
```

```
cam.start_preview()  
cam.annotate(str(datetime.now()))  
cam.take_photo(f"{home_dir}/Desktop/textOnPhoto.jpg")  
cam.stop_preview()
```

Black and White Photography

Capture images in greyscale mode for classic monochrome photos.

```
from picamzero import Camera
from time import sleep
import os
home_dir = os.environ['HOME']
cam = Camera()
cam.greyscale = True
cam.start_preview()
sleep(2)
cam.take_photo(f"{home_dir}/Desktop/bnw.jpg")
cam.stop_preview()
```

Exposure, Gain and Contrast Control

Fine-tune camera settings for optimal image quality.

```
from picamzero import Camera
from time import sleep
cam = Camera()
# Alter the image levels
cam.gain = 1.0
cam.contrast = 2
cam.exposure = 1000000
# Wait a second for changes to take effect
sleep(1)
# Show altered image
cam.start_preview()
sleep(2)
cam.stop_preview()
```

Brightness Control

Adjust overall image brightness (0.0 to 1.0).

```
from picamzero import Camera
from time import sleep
cam = Camera()
# Alter the image levels
cam.brightness = 0.2
# Wait a second for changes to take effect
sleep(1)
# Show altered image
cam.start_preview()
sleep(2)
cam.stop_preview()
```

White Balance

Compensate for different lighting conditions.

Modes: auto, tungsten, fluorescent, indoor, daylight, cloudy

```
from picamzero import Camera  
from time import sleep
```

```
cam = Camera()
```

```
# Alter the image levels  
cam.white_balance = 'cloudy'
```

```
# Wait a second for changes to take effect  
sleep(1)
```

```
# Show altered image  
cam.start_preview()  
sleep(2)  
cam.stop_preview()
```

OpenCV Install

```
sudo apt update
```

```
sudo apt install python3-opencv
```


Sketch Effect (OpenCV)

Transform photos into pencil sketch-style drawings using advanced OpenCV processing.

 **Note:** This effect is applied after capture and won't show in preview.

```
from picamzero import Camera
from time import sleep
import cv2
import os
```

```
home_dir = os.environ['HOME']
cam = Camera()
```

```
sleep(2)
```

```
rgb_array = cam.capture_array()
img = cv2.cvtColor(rgb_array, cv2.COLOR_RGB2BGR)
```

```
# Convert to sketch
greyscale = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
inverted = 255 - greyscale
blur_inverted = cv2.GaussianBlur(inverted, (125, 125), 0)
inverted_blur = 255 - blur_inverted
sketch = cv2.divide(greyscale, inverted_blur, scale=256)
cv2.imwrite(f'{home_dir}/Desktop/sketchImage.jpg', sketch)
```